James Stallkamp

CS 427 HW 13

1)

RSA is based on the fact that factoring a large composite number N into its prime factors p and q is computationally hard. However, if we know φ(N) and N, we can actually factor N easily.

the two equations we have:

φ(N) = (p-1)(q-1)

N = p*q

We can use the first equation to get one of the prime factors, say p, in terms of N and φ(N):

p = N / q

φ(N) = (p-1)(q-1)

Substituting p with N/q in the second equation, we get:

φ(N) = ((N/q)-1)(q-1)

Expanding and simplifying, we get:

φ(N) = (N - (p+q) + 1)

Since we know N and φ(N), we can solve for (p+q) using the above equation, and then use p = N/q to get p and q.

```
def factor_rsa(N, phi):
    s = gmpy2.isqrt((N-1)**2 - 4*N*(phi))
    p = ((N-1) - s) // 2
    q = ((N-1) + s) // 2
    return p, q
```

This function takes as input the RSA modulus N and its Euler's totient φ(N), and returns the two prime factors p and q.

Let's use this function to factor the given 2048-bit RSA modulus:

N =
133140272889335192922108409260662174476303831652383671688547009484253235940586917140482669182256368285260992829447207980183170174867620358952230969986447559330583492429636627298640338596531894556546013113154346823212271748927859647994534586133553218022983848108421465442089919090610542344768294481725103757222421917115971063026806587141287587037265150653669094323116686574536558866591647361053311046516013069669036866734126558017744393751161611219195769578488559882902397248309033911661475005854696

820021069072502248533328754832698616238405221381252145137439919090800085955274389382721844956661113874509547200576180 7

phi = 1331402728893351929221084092606621744763038316523836716885470094842532359405869171404826691822563682852609928294472079801831701748676203589522309699864475593305834924296366272986403385965318945565460131131543468232122717489278596479945345861335532180229838481084214654420899190906105423447682944817251037572149322920465388672184976352567722273701090667853120965897796223554954190060499745678951896873181104980586923156308566936720693205290623996815635903820151773229097447493307026079314281541837265520045272019562263968355003467790624942596389831911789150278351345277516070178590645117315204402981816860178885028680

2)

If we have x^2 ≡N y^2, then we can factor N efficiently using the greatest common divisor (GCD) algorithm. The basic idea is to compute the GCD of N and x+y, and the GCD of N and x-y. Since x !=N ±y, neither x+y nor x-y is a multiple of N.

Let d = gcd(N, x+y) and e = gcd(N, x-y). If d or e is a nontrivial factor of N, then we have successfully factored N. Otherwise, we compute f = gcd(d, e). If f is a nontrivial factor of N, then we have successfully factored N. Otherwise, we repeat the process with a new pair of x and y.

To see why this works, note that x^2 ≡N y^2 implies that N divides (x+y)(x-y). If N and (x+y) share a nontrivial factor d, then d also divides (x+y)^2 - N = x^2 + 2xy + y^2 - N = 2xy, which means that d divides x and/or y. Similarly, if N and (x-y) share a nontrivial factor e, then e divides x and/or -y. By computing the GCD of (x+y) and N, and the GCD of (x-y) and N, we can detect any factors of N that are also factors of (x+y) or (x-y), respectively.

The running time of this algorithm is proportional to the number of pairs of x and y that we need to test. Since there are at most two square roots of any element mod N, there are at most four distinct pairs (x,y) that satisfy x^2 ≡N y^2. Therefore, the expected running time of the algorithm is O(1). In practice, however, the algorithm may take longer due to implementation details and the size of the input.